ndexes of Subarray Sum

ifficulty: **Medium** Accuracy: **16.5%** Submissions: **1.9M** Points: **4** Average Time: **20m**

Given an array **arr[]** containing only non-negative integers, your task is to find a continuous subarray (a contiguous sequence of elements) whose sum equals a specified value **target**. You need to return the **1-based indices** of the leftmost and rightmost elements of this subarray. You need to find the first subarray whose sum is equal to the target.

Note: If no such array is possible then, return [-1].

Examples:

Input: arr[] = [1, 2, 3, 7, 5], target = 12
Output: [2, 4]
Explanation: The sum of elements from 2nd to 4th position is 12.

Input: arr[] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], target = 15
Output: [1, 5]
Explanation: The sum of elements from 1st to 5th position is 15.

Input: arr[] = [5, 3, 4], target = 2
Output: [-1]
Explanation: There is no subarray with sum 2.

```java
class Solution {
    static ArrayList<Integer> subarraySum(int[] arr, int target) {

        ArrayList<Integer> result = new ArrayList<>();

        int start = 0;
        int sum = 0;

        for (int end = 0; end < arr.length; end++) {
            sum += arr[end];

            while (sum > target && start <= end) {
                sum -= arr[start];
                start++;
            }

            if (sum == target) {
                result.add(start + 1); // 1-based index
                result.add(end + 1);   // 1-based index
                return result;
            }
        }

        result.add(-1);
        return result;
    }
}
```

Missing in Array

Difficulty: **Easy** Accuracy: **29.59%** Submissions: **1.6M** Points: **2** Average Time: **15m**

You are given an array **arr[]** of size **n - 1** that contains **distinct integers** in the range from 1 to n (inclusive). This array represents a permutation of the integers from 1 to n with **one element missing**. Your task is to identify and return the **missing element**.

Examples:

Input: arr[] = [1, 2, 3, 5]
Output: 4
Explanation: All the numbers from 1 to 5 are present except 4.

Input: arr[] = [8, 2, 4, 5, 3, 7, 1]
Output: 6
Explanation: All the numbers from 1 to 8 are present except 6.

Input: arr[] = [1]
Output: 2
Explanation: Only 1 is present so the missing element is 2.

```java
class Solution {
    int missingNum(int arr[]) {

        int n = arr.length + 1;
        int xor = 0;

        for (int i = 1; i <= n; i++) {
            xor ^= i;
        }


        for (int i = 0; i < arr.length; i++) {
            xor ^= arr[i];
        }

        return xor;
    }
}
```

Second Largest

Difficulty: **Easy** Accuracy: **26.72%** Submissions: **1.5M** Points: **2** Average Time: **15m**

Given an array of **positive** integers **arr[]**, return the **second largest** element from the array. If the second largest element doesn't exist then return **-1.**

Note: The second largest element should not be equal to the largest element.

Examples:

Input: arr[] = [12, 35, 1, 10, 34, 1]
Output: 34
Explanation: The largest element of the array is 35 and the second largest element is 34.

Input: arr[] = [10, 5, 10]
Output: 5
Explanation: The largest element of the array is 10 and the second largest element is 5.

Input: arr[] = [10, 10, 10]
Output: -1
Explanation: The largest element of the array is 10 and the second largest element does not exist.

```java
class Solution {
    public int getSecondLargest(int[] arr) {
        int largest = -1;
        int secondLargest = -1;

        for (int num : arr) {
            if (num > largest) {
                secondLargest = largest;
                largest = num;
            }
            else if (num < largest && num > secondLargest) {
                secondLargest = num;
            }
        }

        return secondLargest;

    }
}
```